B.TECH DATA SCIENCE

Module 5: Big Data, Visualization and Applications

B.Tech Data Science — Expanded Study Notes and Exam Preparation

Contents
Four chapters of expanded concepts, definitions, examples, applications, methods and revision questions.
Includes side headings on every study page and important questions at the end for exam preparation.

Prepared for learning and exam revision. Follow your university syllabus and faculty guidance where they differ.

1.1 Big Data Fundamentals

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Definition

Big data describes datasets whose volume, speed, variety or complexity challenges conventional data-processing approaches. The term depends on the tools, time limits and reliability requirements of the task.

Why it matters

Data from online services, sensors, transactions, scientific instruments and connected devices can grow rapidly. Distributed storage and processing can help manage scale when a single machine is insufficient.

The 5 Vs

Volume is the amount of data; velocity is the rate of generation and processing; variety is the range of formats; veracity concerns quality and trustworthiness; value concerns useful outcomes.

Distributed computing

Distributed computing divides storage or work across multiple machines. It can improve capacity and throughput, but introduces network communication, coordination, monitoring and failure-handling challenges.

Big data is not only size

A smaller dataset with strict real-time requirements or complex formats may also be difficult to process. The solution should fit the workload rather than adopt distributed tools automatically.

1.2 The 5 Vs in Detail

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Volume

Volume refers to the amount of data stored or processed. Strategies include compression, partitioning, efficient file formats, retention policies and distributed storage.

Velocity

Velocity concerns how quickly data arrives and how quickly results are required. Batch processing handles collected groups of records; streaming processes events continuously or in small increments.

Variety

Variety includes relational tables, logs, text, images, audio, video and sensor records. Integrating different formats requires shared identifiers, definitions and transformation rules.

Veracity

Veracity covers accuracy, completeness, consistency and uncertainty. Large volume does not compensate for unreliable data, so quality checks and provenance are essential.

Value

Value is the benefit gained from processing data. A project should identify a useful question and measure whether the results improve a decision, reduce cost or provide insight.

1.3 Distributed Storage and Fault Tolerance

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Distributed storage

Distributed storage keeps data across multiple machines. It can increase capacity and availability, but data placement, replication, permissions and consistency must be managed.

Partitioning

Partitioning divides a dataset into pieces using a key or strategy. Good partitioning can improve parallel work, while uneven partitions may create a bottleneck known as a hot partition.

Replication

Replication stores copies of data on different nodes or locations. It can improve availability and recovery but consumes extra storage and requires rules for updates.

Fault tolerance

Fault tolerance is the ability to continue operating or recover when components fail. Techniques include replication, retries, checkpointing, monitoring and recovery procedures.

Trade-offs

Distributed systems must balance speed, consistency, availability, cost and complexity. The right design depends on how the data is used and what failures are acceptable.

1.4 Batch and Stream Processing

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Batch processing

Batch processing collects records and processes them together on a schedule or when a batch is ready. It is suitable for daily reports, historical analysis and large jobs without strict immediate response needs.

Stream processing

Stream processing handles events continuously or in small windows. It supports near-real-time alerts, fraud monitoring, sensor dashboards and event-driven systems.

Latency and throughput

Latency is the delay before a result is produced. Throughput is the amount of work completed per unit of time. A system may have high throughput but still have unsuitable latency for an interactive application.

Windowing

Streaming systems may aggregate events over time windows, such as counts per minute. Late events, duplicate events and event-time versus processing-time choices must be handled explicitly.

Selection

Choose batch, streaming or a combination according to business need, event volume, freshness requirements, reliability and operating cost.

2.1 Hadoop Ecosystem

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Overview

Hadoop is an ecosystem associated with distributed storage and batch processing of large datasets. Its components support storage, resource coordination and parallel data processing.

HDFS

The Hadoop Distributed File System stores large files across a cluster. Files are divided into blocks and blocks can be replicated according to the system configuration.

NameNode and DataNodes

At a high level, the NameNode manages filesystem metadata, while DataNodes store blocks. Exact responsibilities and resilience features depend on the Hadoop deployment and version.

YARN

YARN coordinates cluster resources and application execution in Hadoop environments. It separates resource management from the details of individual processing frameworks.

Use cases and limitations

Hadoop-style systems can suit large batch workloads. They may be less suitable for tiny files, low-latency interactive tasks or situations where cluster administration costs outweigh the benefit.

2.2 HDFS: Storage Concepts

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

File blocks

HDFS divides large files into blocks distributed across DataNodes. Block size and replication are configuration choices that influence storage and processing.

Replication and recovery

Replicated blocks can help preserve availability when a node fails. The system can detect missing replicas and restore them according to configured policies.

Data locality

Processing near the machines holding the data can reduce network transfer. Frameworks may schedule work with data locality in mind, though exact placement is not always possible.

Reliability

Distributed storage still needs monitoring, capacity planning, permissions, backups where appropriate and recovery testing. Replication is not a substitute for every form of backup.

File suitability

HDFS is designed around large files and streaming access patterns. Large numbers of tiny files can create metadata overhead and reduce efficiency.

2.3 MapReduce Processing

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Map stage

The map stage processes input records and emits intermediate key-value pairs. The mapping logic depends on the problem.

Shuffle and sort

Intermediate outputs are grouped by key and transferred to the appropriate reducers. This stage involves network communication and can become a performance bottleneck.

Reduce stage

The reduce stage processes all values associated with a key to produce summaries or results. The operation might sum counts, combine values or compute another aggregation.

Word-count example

A mapper emits (word, 1) for each word. The framework groups values by word, and a reducer adds the values to produce each word's total count.

Strengths and limits

MapReduce provides a model for parallel batch processing and recovery, but multi-stage jobs may involve disk I/O and can be less convenient for iterative interactive workloads.

2.4 Apache Spark and Tool Selection

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Spark overview

Apache Spark is a distributed processing engine that supports batch processing, SQL, streaming-related workloads and machine-learning tasks through its ecosystem.

In-memory processing

Spark can keep intermediate data in memory when useful, which may improve iterative workloads. Memory limits, disk spill, network transfer and cluster configuration still affect performance.

Hadoop and Spark

Hadoop is an ecosystem that includes HDFS and resource-management components; MapReduce is a processing model. Spark is a processing engine and can operate with different storage and cluster environments.

Tool selection

Consider data size, latency, processing pattern, skills, reliability, security, cost and existing infrastructure. Newer or more complex tools are not automatically better.

Operational concerns

Distributed processing requires observability, dependency management, capacity planning, access control and a clear strategy for failed jobs.

3.1 Principles of Data Visualization

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Purpose

Data visualization presents data visually to help people compare values, understand patterns, identify exceptions and communicate findings.

Chart choice

Use bar charts for category comparisons, line charts for trends over time, histograms for numerical distributions, scatter plots for relationships and maps for meaningful geographic patterns.

Visual hierarchy

Use titles, labels, units, readable fonts and consistent colour meaning. Make the central finding easy to identify without making the chart misleading.

Accessibility

Do not rely on colour alone to convey meaning. Use sufficient contrast, clear labels and designs that remain understandable for viewers with colour-vision differences.

Avoid clutter

Remove unnecessary 3D effects, excessive gridlines, decorative icons and labels that compete with the data. Simpler charts often communicate more accurately.

3.2 Common Charts and Their Uses

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Bar chart

A bar chart compares categories. Start a bar axis at zero when the length encodes magnitude, unless a clearly justified alternative is used and the visual remains honest.

Line chart

A line chart is appropriate for ordered time data when connecting observations makes sense. Label time intervals and avoid implying continuous measurements where the data is only categorical.

Histogram

A histogram shows how numerical values are distributed across bins. Bin width changes the apparent pattern, so consider multiple reasonable settings when exploring data.

Scatter plot

A scatter plot reveals relationships, clusters and outliers between numerical variables. A trend line may summarise association but does not prove causation.

Box plot and heatmap

Box plots compare distributions across groups. Heatmaps encode values through colour in a matrix, such as correlation values or activity by day and hour; use a clear scale and labels.

3.3 Misleading Visuals and Dashboard Design

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Axis manipulation

A truncated axis can exaggerate small differences in a bar chart. Axis choices should suit the chart type and be clearly visible.

Cherry-picking

Showing only a convenient time period or subgroup can create a false impression. State the time range, sample and relevant exclusions.

Aggregation effects

Averages can hide variation and subgroup differences. Show distributions or breakdowns when they materially affect interpretation.

Dashboard layout

Group related metrics, put the most important information first and provide filters only when they help answer a real question. Too many visual elements increase cognitive load.

Report narrative

A useful report states the question, data source, method, findings, uncertainty, limitations and recommended next steps. Separate facts from interpretation.

3.4 Communicating Uncertainty and Findings

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Uncertainty

Data may contain sampling variation, measurement error, missingness and model uncertainty. State which uncertainty matters for the conclusion.

Context

A percentage should usually be accompanied by its denominator and time period. A change from 1 to 2 cases is a 100% increase but may not imply a large operational problem.

Comparison

Use consistent units, definitions and periods when comparing groups. Adjusted comparisons may be needed when the groups differ in important ways.

Audience adaptation

Executives may need impact and recommended action; technical reviewers may need assumptions, code, validation and limitations. The evidence must remain consistent across versions.

Actionable conclusion

Conclude with what the evidence supports, what remains uncertain and what should happen next. Avoid overclaiming or confusing correlation with cause.

4.1 Data Science Applications

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Healthcare

Applications include risk estimation, resource planning, medical-image support and monitoring. Data privacy, representativeness, clinical validation and professional oversight are critical.

Finance

Fraud detection, credit-risk assessment and transaction monitoring use patterns in financial records. Systems need to handle changing behaviour and consider the consequences of false alarms.

Retail

Retail analytics supports demand forecasting, inventory management, product recommendations and customer analysis. Seasonality, stockouts and changing promotions affect interpretation.

Agriculture

Weather, soil, satellite imagery and sensor data can support crop monitoring, irrigation planning and yield estimation. Missing sensor data and local context can limit reliability.

Transport and education

Transport analytics may forecast delays and optimise routes. Education analytics may monitor learning outcomes and allocate support, with safeguards against unfair labelling or misuse.

4.2 Deployment Patterns and MLOps

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Batch deployment

A batch job generates predictions on a schedule, such as a daily demand forecast. It is often simpler than a real-time service when immediate response is unnecessary.

API deployment

An API accepts inputs and returns model outputs to another application. It needs input validation, authentication, latency targets, error handling and capacity management.

Dashboards and embedded systems

Dashboards display metrics and forecasts to decision-makers. Embedded systems may use predictions within devices or operational processes and can have strict resource constraints.

Monitoring

Monitor data quality, input drift, output patterns, latency, failure rates and prediction quality when outcomes become available. Keep logs sufficient for debugging while respecting privacy.

Model lifecycle

Version the data, code and model. Define approval, rollout, rollback, incident response and retraining procedures before a model becomes operationally important.

4.3 Mini-Project Methodology

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Step 1 — Define a question

Choose a focused question with a measurable outcome, realistic scope and identifiable audience. State what decision the project may inform.

Step 2 — Obtain and understand data

Select appropriate data, confirm permission and document its source, time period and definitions. Inspect rows, columns, types and missingness.

Step 3 — Clean and explore

Resolve or document duplicates, missing values, invalid records and unusual values. Use summary statistics and charts to understand patterns and limitations.

Step 4 — Build a baseline and method

Start with a simple benchmark. Choose a method suited to the task, keep evaluation data separate and avoid leakage during feature preparation.

Step 5 — Evaluate and communicate

Use suitable metrics, inspect error cases, explain limitations and present findings. Include enough detail that another student can understand the workflow.

4.4 Example Mini-Projects

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Student performance analysis

Explore attendance, assessment completion and results to identify aggregate patterns. Avoid exposing personal records and do not interpret association as proof of cause.

Electricity demand forecasting

Use historical demand and time-related variables to forecast future usage. Use time-ordered evaluation and compare with a simple baseline such as the previous period.

Spam classification

Prepare labelled messages, split data carefully, train a classifier and evaluate precision and recall. Protect message content and inspect false positives.

Crop-yield analysis

Combine yield records with weather, soil or management variables where permitted. Check differences across regions and seasons and clearly state missing factors.

Transport-delay analysis

Explore delay patterns by route, time and conditions. Validate timestamp consistency and report uncertainty rather than treating a single average as the whole story.

5. Important Exam Questions and Revision

Study focus: Big Data, Visualization and Applications. Read the side headings, understand the example, and revise the key ideas.

Long-answer questions

1. Explain the 5 Vs of big data. 2. Describe HDFS and MapReduce at a high level. 3. Compare batch and stream processing. 4. Explain how to choose an appropriate chart. 5. Discuss data science applications in four domains. 6. Explain the stages of a data science mini-project and model monitoring.

Short-answer questions

1. Define fault tolerance. 2. What is data locality? 3. Define partitioning and replication. 4. What is data drift? 5. State two uses of a dashboard. 6. Differentiate throughput and latency.

Case-based questions

1. Choose a processing approach for real-time fraud alerts and justify it. 2. Recommend charts for monthly sales, category counts and numerical distributions. 3. Design a mini-project to forecast electricity demand. 4. Explain safeguards for a healthcare analytics deployment.

Revision checklist

Review 5 Vs, distributed computing, fault tolerance, HDFS, MapReduce, Spark, batch versus streaming, chart selection, misleading visuals, applications, deployment and mini-project workflow.

Exam tip

Use a labelled diagram for architecture or workflow questions. Explain each component's role, then discuss advantages, limitations and a relevant application.